

FALL 2009: 2-11-2009
BILKENT UNIVERSITY
Department of Electrical and Electronics Engineering
EEE102 Introduction to Digital Circuit Design
Midterm Exam I - SOLUTIONS

Surname: _____
Name: _____
ID-Number: _____
Signature: _____

Duration is 120 minutes. Solve all 7 questions. Show all your work.
No books, notes, or calculators.

Q1 (15 points)	
Q2 (15 points)	
Q3 (10 points)	
Q4 (18 points)	
Q5 (15 points)	
Q6 (12 points)	
Q7 (15 points)	
Total	

Q1. (15 points) (a) Perform subtraction on the given unsigned binary numbers using the 2's complement of the subtrahend. You should show all of your arithmetic operations in binary form. Also, give your final answer in decimal representation.

(i) (1.5 pts) $10011 - 10001 = (+2)_{10}$

1111110 (carries)	10001 → (2's complement) 101111
010011	
101111	
+-----	
1 000010	

(ii) (1.5 pts) $100010 - 100011 = (-1)_{10}$

00000000 (carries)	0100011 → (2's complement) 1011101
0100010	
1011101	
+-----	
1111111	1111111 → (2's complement) 0000001

(iii) (1.5 pts) $1001 - 101000 = (-31)_{10}$

00110000 (carries)	0101000 → (2's complement) 1011000
0001001	
1011000	
+-----	
1100001	0011110 → (2's complement) 0011111

(b) Perform the following conversions:

(i) (1.5 pts) $4675542_8 = (137B62)_{16} = (0001\ 0011\ 0111\ 1011\ 0110\ 0010)_2$

(ii) (1.5 pts) $1F08B725_{16} = (3702133445)_8 = (000\ 011\ 111\ 000\ 010\ 001\ 011\ 011\ 100\ 100\ 101)_2$

(iii) (1.5 pts) $43.125_{10} = (101011.001)_2$

<u>Whole</u>	<u>Fraction</u>
$0.125 \times 2 = 0$ (MSB)	0.25
$0.25 \times 2 = 0$	0.5
$0.5 \times 2 = 1$ (LSB)	0

(c) (6 pts) Convert the BCD number $100010010101.001101110101_{10}$ to binary and (ordinary) decimal form (Note that the subscript $_{10}$ indicates BCD).

Decimal Form = 895.375 (Whole Part of the number found using repeated divisions by 2, and using the remainder as LSB to MSB)

Binary Form = 1101111111.011

<u>Whole</u>	<u>Fraction</u>
$0.375 \times 2 = 0$ (MSB)	0.75
$0.75 \times 2 = 1$	0.5
$0.5 \times 2 = 1$ (LSB)	0

Q2. (15 points) (a) Simplify the following Boolean expressions to the indicated number of literals. You need to state the name of the Theorems used in every step of the simplification process. (Note: $x+x'yz$ is a 4 literal expression, since x and x' are counted as 2 literals):

(i) (3 pts) $A'B(D' + C'D) + B(A + A'CD)$ to one literal
 $= A'BD' + A'BC'D + AB + A'BCD$ (Distributivity) $= A'BD' + AB + A'BC'D + A'BCD$
 (Commutativity) $= A'BD' + AB + A'BD$ (Combining) $= A'B + AB$ (Combining) $= \underline{B}$
 (Combining)

(ii) (3 pts) $(x'y' + z)' + z + xy + wz$ to three literals
 $= (x+y).z' + z + xy + wz$ (DeMorgan's) $= xz' + yz' + xy + z + wz$ (Distributivity & Associativity)
 $= xz' + yz' + xy + z$ (Covering) $= xz' + xy + yz' + z$ (Associativity)
 $= xz' + xy + y + z$ (Proof Below) $= (xz' + z) + (xy + y)$ (Associativity) $= x + z + y$ (Covering Theorem plus Proof below for $xz' + z = x + z$)

Proof of $X + X'Y = X + Y$

$X.1 + X'Y + Y.1 = X.1 + X'Y$ (Consensus Theorem) $= X + X'Y$ (Identities Theorem)

So, working with the left-hand side, we get:

$X.1 + X'Y + Y.1 = X + X'Y + Y$ (Identities)
 $= X + Y.(X' + 1)$ (Distributivity Theorem)
 $= X + Y$ (Null Elements)

Therefore $X + X'Y = X + Y$

(b) (3 pts) Determine the dual of the following Boolean expression, introducing

parentheses as needed: $(x_1 + x_2x_3x_4)'$

$f_D = x_1' + x_2'x_3'x_4'$

(c) A Boolean function is self-dual if $f(X) = f^d(X)$ - that is, if the function and its dual are the same. Determine whether the following Boolean expressions are selfdual:

(i) a (ii) ab (iii) $ab + (ab)'$

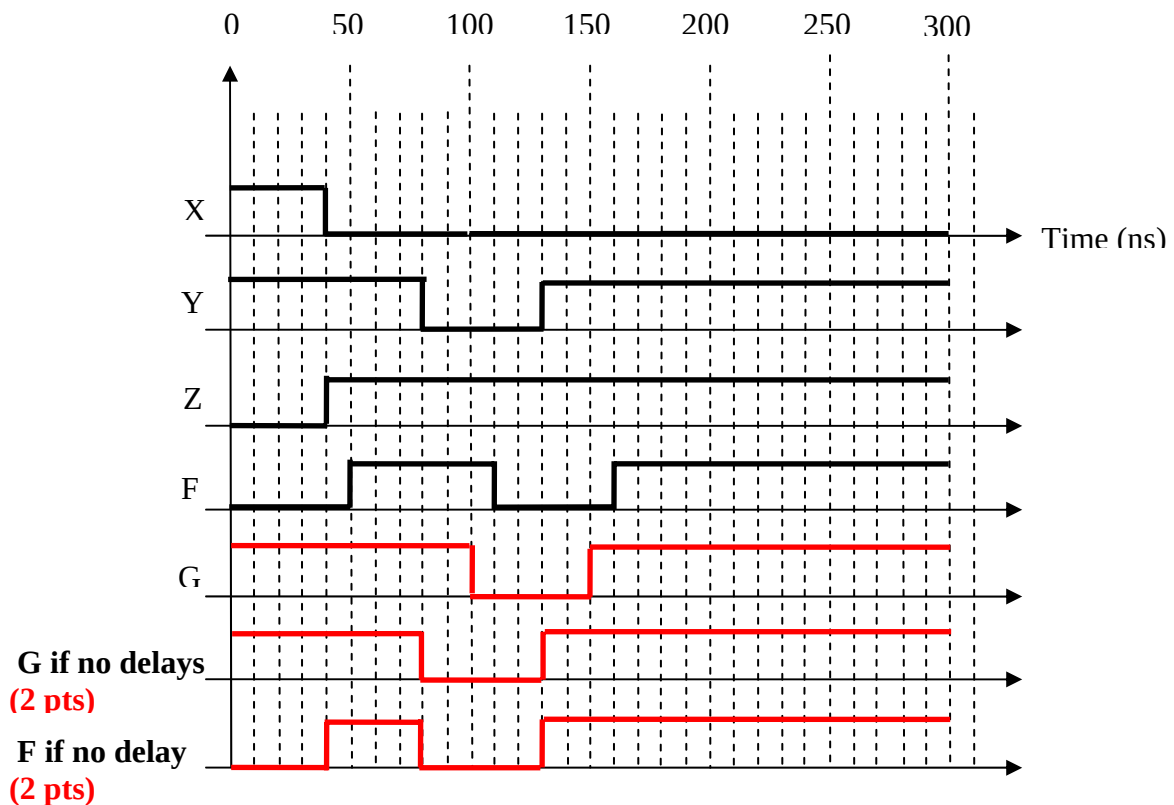
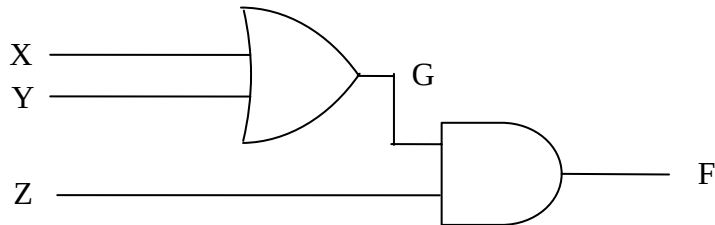
Note: You need to show all of your work for full-credit.

(i) Self-dual ($f_D = a = f$) (2 pts)

(ii) Not self-dual ($f_D = a+b \neq f$) (2 pts)

(iii) Not self-dual ($f_D = (a+b).(a'+b') \neq f$) (2 pts)

Q3. (10 points) For the circuit given below, a timing simulation is performed and the input and output waveforms are given. The two gates in the circuit have different amounts of delay. Find the delays of each gate and draw G. Explain and justify your answer. Also draw G and F if the gates do not have any delays.



**OR Gate = 20 ns (3 pts),
AND Gate = 10 ns (3 pts).**

Q4. (18 points) (a) Express the complement of the following functions in sum-of-minterms form:

(i) (4 pts) $F(A, B, C, D) = \sum(3, 5, 9, 11, 15)$

$$F' = \sum(0, 1, 2, 4, 6, 7, 8, 10, 12, 13, 14) = A'B'C'D' + A'B'C'D + A'BC'D' + A'BCD' + A'BCD + AB'C'D' + AB'CD' + ABC'D' + ABCD'$$

(ii) (4 pts) $F(A, B, C, D) = \prod(2, 4, 5, 7)$

$$F'(A, B, C, D) = \sum(2, 4, 5, 7) = A'B'CD' + A'BC'D' + A'BC'D + A'BCD$$

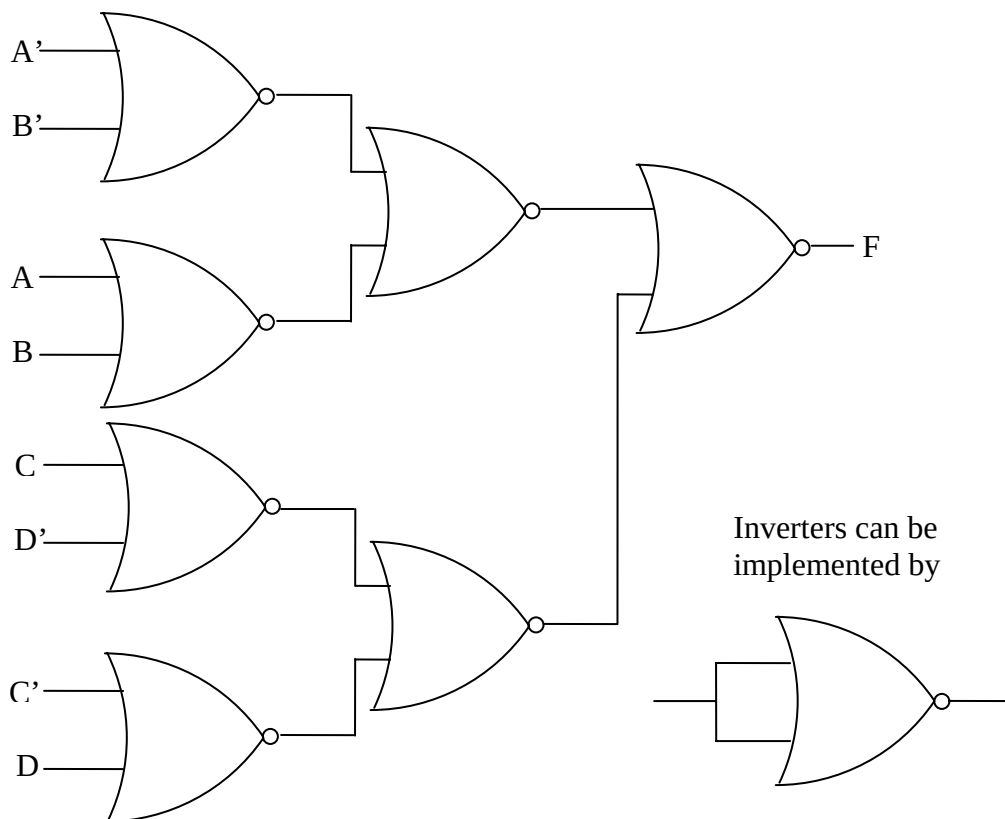
(b) For both parts given below, you should assume that the signals are only available to you in their standard form, and to get their complemented version, you need to implement inverters using NOR gates.

(i) (3 pts) Draw a logic diagram using only two-input NOR gates to implement the following function:

$$F(A, B, C, D) = \overline{(A \oplus B)}(C \oplus D) = (A \oplus B)'(C \oplus D)$$

$$\overline{(A \oplus B)} = AB + A'B'$$

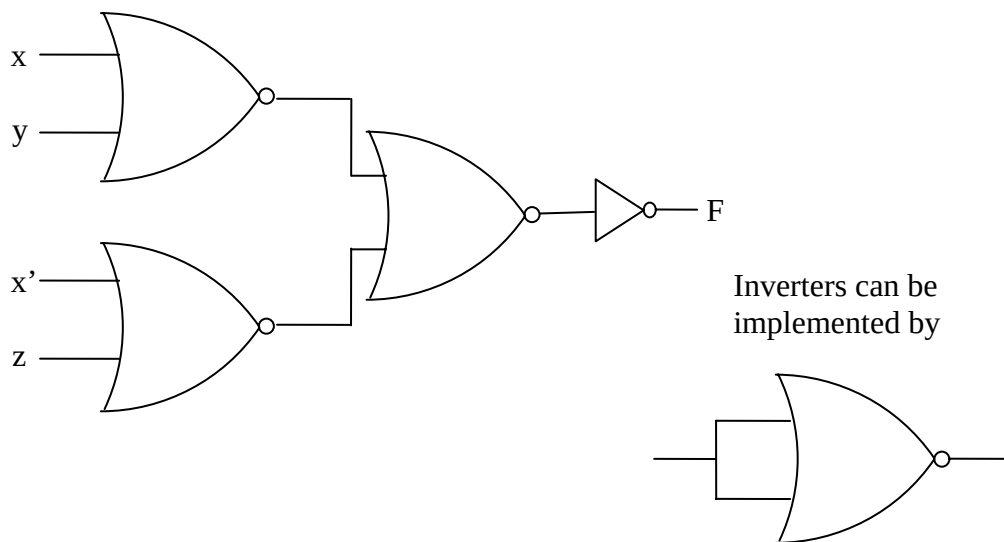
$$(C \oplus D) = C'D + CD'$$



(ii) (3 pts) Implement the following function with two-level NOR gates and draw the circuit. (You may simplify the function before implementation.)

$$F(x, y, z) = \overline{(x + y)(x' + z)} = ((x + y)(x' + z))'$$

$$F = x'y' + xz'$$



(c) (4 pts) We define the DISAGREE function $z(x_1, x_2, \dots, x_n)$ to be 1 if and only if $x_i \neq x_j$ for some i and j . Determine whether or not {DISAGREE} is functionally complete. Prove your answer. (In this context “functionally complete” means “being able to implement any combinational circuit”.)

If we can use Disagree to implement NAND or NOR, then Disagree is functionally complete.

No, “Disagree” is not functionally complete, since if we tie all of its inputs but two of them (for example x_1 and x_2) to ‘0’ (or setting $n = 2$), then the output is the XOR of the 2 inputs. And, with XOR, we can implement “NOT”, but not “AND” or “OR” gates.



Q5. (15 points) The following VHDL code is given for modules “top”, “aaa”, and “bbb”. Draw the circuit schematic of module “top” using “aaa” and “bbb” as components. Find out and explain what “aaa” and “bbb” do and finally state what the function of “top” is.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity top is
  Port ( x : in  STD_LOGIC_VECTOR (2 downto 0);
        y : out STD_LOGIC_VECTOR (2 downto 0));
end top;

architecture Behavioraltop of top is
  signal w:std_logic_vector(2 downto 0);
  component aaa
    Port ( a : in  STD_LOGIC;
          b : in  STD_LOGIC;
          c : in  STD_LOGIC;
          z : out STD_LOGIC_VECTOR (2 downto 0));
  end component;
  component bbb
    Port ( d : in  STD_LOGIC_VECTOR (2 downto 0);
          z : out STD_LOGIC_VECTOR (2 downto 0));
  end component;
begin
  label1: aaa port map(x(2),x(1),x(0),w);
  label2: bbb port map(w,y);
end Behavioraltop;
```

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity aaa is
  Port ( a : in  STD_LOGIC;
        b : in  STD_LOGIC;
        c : in  STD_LOGIC;
        z : out STD_LOGIC_VECTOR (2 downto 0));
end aaa;

architecture Behaviorala of aaa is
begin
  z(2)<= not a;
  z(1)<= not b;
  z(0)<= not c;
end Behaviorala;
```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity bbb is
    Port ( d : in  STD_LOGIC_VECTOR (2 downto 0);
          z : out STD_LOGIC_VECTOR (2 downto 0));
end bbb;

architecture Behavioralb of bbb is
    signal x,y:std_logic_vector(3 downto 0);
begin
    y<=x+1;
    z(2)<=y(2);
    z(1)<=y(1);
    z(0)<=y(0);
    x(3)<='0';
    x(2)<=d(2);
    x(1)<=d(1);
    x(0)<=d(0);
    -- Note that + operator is addition and it works for std logic vectors.
    -- However the person who wrote the code is not sure about what happens when '1'
    -- is added to "111".
end Behavioralb;

```

(5 pts) aaa: takes the ones complement of a number

(5 pts) bbb: adds 1 to the number

(5 pts) top: finds the 2's complement for 3-bit 2's complement numbers

Q6. (12 points) Write a VHDL code using behavioral-type programming to implement an 84-2-1 to BCD code converter. The 84-2-1 code system is shown in the table below. If the input is other than the 84-2-1 codes shown below then the output should be the binary equivalent of the hexadecimal number E (meaning Error).

Decimal Digit	8, 4, -2, -1 Code	BCD Code
0	0000	0000
1	0111	0001
2	0110	0010
3	0101	0011
4	0100	0100
5	1011	0101
6	1010	0110
7	1001	0111
8	1000	1000
9	1111	1001

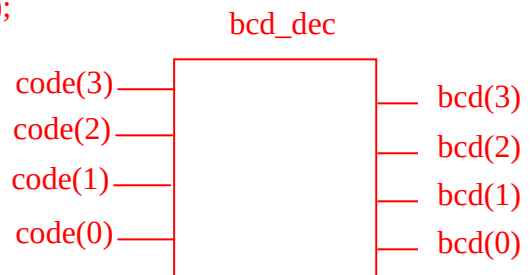
```
library ieee;
library ieee.std_logic_1164.all;
```

```
entity bcd_dec is
    port( code: in std_logic_vector (3 downto 0);
          bcd: out std_logic_vector (3 downto 0) );
end bcd_dec;
```

```
architecture bcddecoder of bcd_dec is
```

```
    process(code)
    begin
        case CONV_INTEGER(code) is
            when 0 => bcd <= "0000";
            when 7 => bcd <= "0001";
            when 6 => bcd <= "0010";
            when 5 => bcd <= "0011";
            when 4 => bcd <= "0100";
            when 11 => bcd <= "0101";
            when 10 => bcd <= "0110";
            when 9 => bcd <= "0111";
            when 8 => bcd <= "1000";
            when 15 => bcd <= "1001";
```

```
            when others => bcd <= "0111";
        end case;
    end process;
end bcddecoder;
```

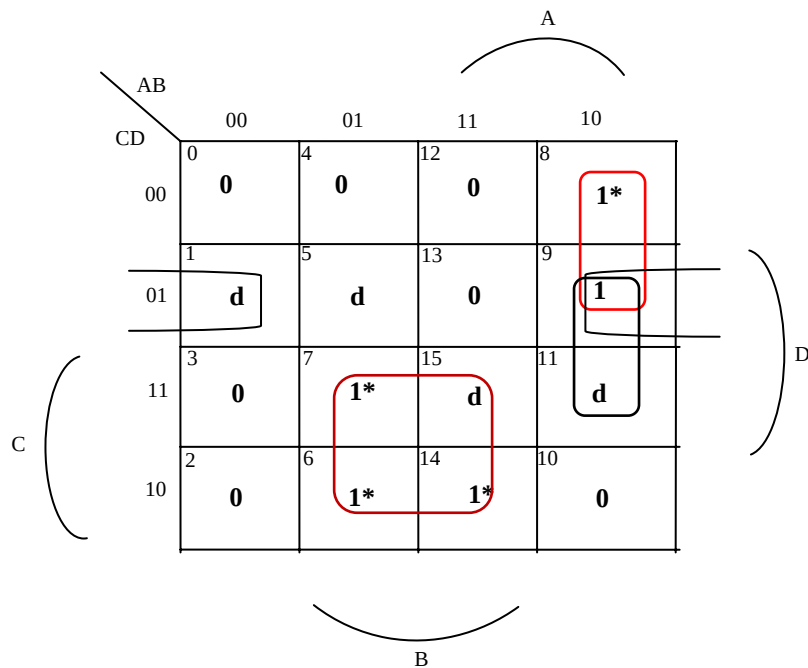


Q7. (15 points) Given the function $Z(A, B, C, D) = \sum (6, 7, 8, 9, 14) + d(1, 5, 11, 15)$

- (3 pts)** Find all minimal sums for F
- (6 pts)** Find all minimal products for F
- (3 pts)** Which of the above minimal sums are equivalent to which of the above minimal products?
- (3 pts)** Which of the above found expressions would you use for implementation. Why?

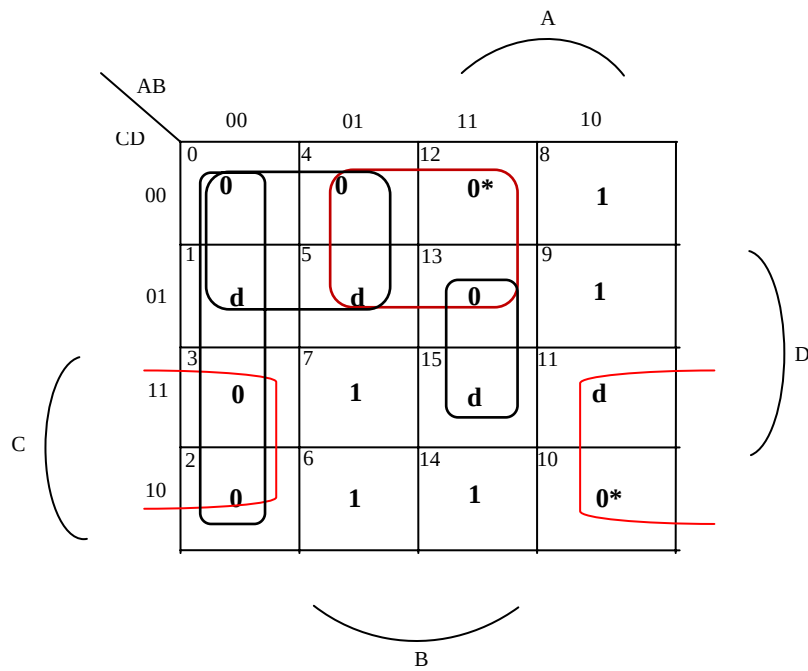
Solution:

(a)



$F_1 = BC + AB'C'$ There is only one minimal sum

(b)



$$F_2 = (B' + C)(B + C')(A + C)$$

$$F_3 = (B' + C)(B + C')(A + B)$$

There are two minimal products

(c) In all solutions 1,5,11 are zero and 15 is one. Therefore they are all equivalent.

(d) I would use F_1 because it has two terms while the others have three terms.

ENTITY DECLARATION

```
entity entity_name is
    generic ( constant_names : constant type;
              constant_names : constant type;
              ...
              constant_names : constant type);
    port ( signal_names : mode signal_type;
           signal_names : mode signal_type;
           ...
           signal_names : mode signal_type);
end entity_name;
```

ARCHITECTURE DEFINITIONS

```
architecture architecture-name of entity-name is
    type declarations
    signal declarations
    constant declarations
    function definitions
    procedure definitions
    component declarations
begin
    concurrent statement
    ...
    concurrent statement
end architecture-name;
```

COMPONENT DECLARATION

```
component component_name
    port ( signal_names : mode signal type;
           signal_names : mode signal type;
           ...
           signal_names : mode signal type);
end component;
```

COMPONENT INSTANTIATION

```
label: component_name port map (signal1, signal2, ..., signaln);
or,
label: component_name port map (port1 => signal1, port2 => signal2, ..., portn => signaln);
```

DATAFLOW TYPE STATEMENTS:

Simple concurrent assignment statement

```
signal_name <= expression;
```

Conditional concurrent assignment statement

```
signal_name <=
    expression when boolean-expression else
    expression when boolean-expression else
    ...
    expression when boolean-expression else
    expression;
```

with-select statement

```
with expression select
    signal_name <= signal_value when choices,
                   signal_value when choices,
    ...
    signal_value when choices;
```

Note that conditional concurrent assignment statement and with-select statement cannot be used in a process statement. Instead, in a process, one can use the sequential conditional assignment statements if and case.

BEHAVIORAL TYPE STATEMENTS:

process statement

process(*signal_name*, *signal_name*, ..., *signal_name*)

type_declarations

variable_declarations

constant_declarations

begin

sequential-statement

 ...

sequential-statement

end process;

Simple sequential assignment statement

signal_name <= *expression*;

Simple variable assignment statement

variable_name := *expression*;

if statement in its general form

if *boolean_expression* then *sequential_statements*

elsif *boolean_expression* then *sequential_statements*

...

elsif *boolean_expression* then *sequential_statements*

else *sequential_statements*

end if;

Note that you may not use the else and/or the elsif.

case-when statement

case *expression* is

 when *choices* => *sequential_statements*

 ...

 when *choices* => *sequential_statements*

end case;

loop statement

loop

sequential_statement

 ...

sequential_statement

end loop;

for-loop statement

for *identifier* in range loop

sequential_statement

 ...

sequential_statement

end loop;

while statement

while *boolean_expression* loop

sequential_statement

 ...

sequential_statement

end loop;

Note that the if, case, loop, for, and while statements are called sequential statements and they can only be used in a process statement. Also note that each process is one concurrent statement.

If the "ieee.std_logic_arith.all" and "ieee.std_logic_unsigned.all" packages are included then + and – operators for addition and subtraction can be used for UNSIGNED binary, SIGNED binary, and STD_LOGIC_VECTOR types.

Concatenation operator & is used as follows: If A and B are 2 bit numbers then A&B is a four bit number with A being more significant.